EXTENDED RTAB-Map AND REDUCED Fast-SCNN FOR AUTONOMOUS INDUSTRIAL VEHICLE NAVIGATION

Azzam Wildan Maulana
Department of Electrical Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
6022241101@student.its.ac.id

Rudy Dikairono
Department of Electrical Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
rudydikairono@its.ac.id

Ahmad Zaini
Department of Electrical and Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
zaini@its.ac.id

Mauridhi Hery Purnomo
Department of Electrical and Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
hery@ee.its.ac.id

Moh Ismarintan Zazuli
Department of Electrical Automation Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
ismarintan@its.ac.id

Muhtadin
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia 60111
muhtadin@ee.its.ac.id

***Abstract*—This research develops a dual-layer navigation system for industrial autonomous vehicles. The first layer, an Extended RTAB-Map, performs global navigation and mapping; the second, a Reduced Fast-SCNN, handles local perception for lane detection. The Extended RTAB-Map introduces a skip connection from hardware odometry to pose fusion. The Reduced Fast-SCNN decreases convolutional channels and replaces pyramid pooling with average pooling. Experiments show an increase in pose update rate from 1 Hz to 50 Hz with centimeter-level accuracy (error < 5 cm). The Reduced Fast-SCNN achieves real-time CPU-only inference below 10 ms.**

***Index Terms*—autonomous vehicle, navigation system, machine learning, graph-based SLAM, lane detection**

I. Introduction

Autonomous vehicles are increasingly vital in industrial logistics, offering improvements in safety, efficiency, and operational precision. Industrial plants often require continuous material transport in constrained and hazardous environments, where human operators are prone to fatigue and error. As reported in [1], night-shift workers are particularly susceptible to micro-sleep episodes lasting 1–2 seconds, which can result in serious accidents. To mitigate these risks, autonomous vehicles can replace or assist human-operated systems, ensuring continuous and safe material handling.

An autonomous navigation system generally consists of two main components: the global navigation system for pose estimation and mapping, and the local navigation system for short-range perception and motion control. The global system requires both accuracy and high update rates to maintain stable control. In this study, we extend the Real-Time Appearance-Based Mapping (RTAB-Map) algorithm [2], a graph-based SLAM framework known for its robustness in visual and LiDAR mapping. However, the default RTAB-Map provides a limited update rate (around 1 Hz) and is heavily dependent on visual odometry, which suffers under lighting variations or occlusions [3]. To overcome these limitations, an *Extended RTAB-Map* is proposed by introducing a skip connection from hardware odometry to pose fusion. Odometry data from encoders and IMU sensors are combined with RTAB-Map's corrected pose using a Kalman Filter [4], improving both update frequency and pose accuracy.

For local navigation, lane or drivable-area detection is essential for maintaining safe motion. Deep learning-based semantic segmentation models like Fast-SCNN [5] are efficient but still computationally heavy for CPU-only systems. Although D-Fast-SCNN [6] improves accuracy using dilation, it increases inference time. To meet real-time industrial demands, we design a *Reduced Fast-SCNN* by lowering convolutional channels and replacing pyramid pooling with average pooling, achieving sub-10 ms inference on CPU. The output segmentation provides a confidence level used to adjust vehicle speed and steering dynamically. By integrating the Extended RTAB-Map and Reduced Fast-SCNN, this research delivers a high-frequency, CPU-efficient, and centimeter-accurate navigation system suitable for industrial autonomous vehicles.

II. Methodology

The methodology of this research is divided into three main parts. The first part discusses the architecture of the Extended RTAB-Map. The second part presents the network design of the Reduced Fast-SCNN. The final part explains the integration of the Extended RTAB-Map and Reduced Fast-SCNN into a unified navigation system for autonomous industrial vehicles.

A. Extended RTAB-Map for Global Navigation System

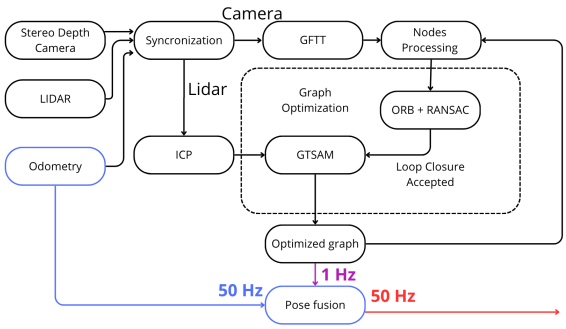

Fig. 1. Block diagram of graph-based SLAM in mapping mode.

The first and most important step is to calculate the hardware odometry using encoder speed and IMU data. The relationship can be expressed as follows:

$$\begin{bmatrix} v_x \\ v_y \\ v_\theta \end{bmatrix} = \frac{1}{\Delta t} \begin{bmatrix} \text{encoder\_to\_meter}\, \text{encoder\_speed}\, \cos\theta \\ \text{encoder\_to\_meter}\, \text{encoder\_speed}\, \sin\theta \\ \text{current\_IMUyaw} - \text{previous\_IMUyaw} \end{bmatrix} \quad (1)$$

Here, v_x represents the linear velocity in the x-direction, v_y is the linear velocity in the y-direction, and v_θ is the angular velocity around the z-axis. The term Δt denotes the time difference between the current and previous readings (20 ms). The variable encoder_to_meter is a conversion factor from encoder speed (in rpm) to meters per second. encoder_speed refers to the encoder output, θ is the yaw angle from the IMU in radians, while current_IMUyaw and previous_IMUyaw denote the consecutive IMU yaw readings in degrees.

The second step is to utilize and optimize the RTAB-Map framework. The first enhancement involves performing approximate time synchronization among camera, LiDAR, and hardware odometry data. This step is essential because each sensor operates at a different update frequency, and desynchronized data could lead to mapping errors.

The second optimization of RTAB-Map involves the use of the GFTT (Good Features to Track) algorithm as an image feature detector. As demonstrated in [7], GFTT provides a fast yet reliable feature extraction performance suitable for real-time applications. The third improvement applies the combination of ORB (Oriented FAST and Rotated BRIEF) and RANSAC (Random Sample Consensus) for feature matching. ORB provides a computationally efficient descriptor while maintaining robustness, and RANSAC filters out outliers from matched keypoints [8]. The output of this step is a loop-closure hypothesis, which indicates that the robot has revisited a previously mapped location. This hypothesis is used to refine and optimize the pose graph.

The fourth enhancement in RTAB-Map involves applying the ICP (Iterative Closest Point) algorithm to establish constraints between the nodes. ICP is utilized to compute the relative pose and covariance between two adjacent nodes [9]. This algorithm is selected because LiDAR laser scans provide higher geometric accuracy compared to depth information from cameras.

The fifth stage involves full graph optimization using all nodes and edges. When a valid loop-closure hypothesis is detected, the corrected pose and covariance are calculated based on the ICP constraint. These values are then used to optimize the entire pose graph through nonlinear optimization using the GTSAM library [10]. This optimization reduces accumulated drift and improves global map consistency.

Despite these improvements, the standard RTAB-Map implementation yields an average pose update rate of only 1 Hz when tested on an Intel i5 12th Gen processor with 16 GB of RAM and no dedicated GPU. To increase the update rate, we introduce a skip connection from the hardware odometry to the pose fusion module (indicated by the blue line in Fig. 1). The pose fusion employs a Kalman Filter to combine the corrected pose and covariance from RTAB-Map with the odometry-based pose and covariance from the hardware sensors. The hardware odometry, calculated from Eq. 1, serves as the velocity model in the Kalman Filter, while the RTAB-Map output acts as the measurement model.

The fusion strategy begins by assigning a low covariance value (around 0.01) to the hardware odometry, prioritizing responsiveness during the vehicle's initial motion. As the system detects more loop closures, the covariance of the RTAB-Map corrected pose gradually decreases, allowing it to dominate the fusion process. Consequently, the final pose estimation becomes both accurate and smooth over time. The use of the Kalman Filter not only improves pose accuracy but also minimizes abrupt transitions during map corrections. The resulting fused pose achieves a 50 Hz update rate on the same hardware setup, with centimeter-level precision suitable for real-time navigation.

B. Reduced Fast-SCNN for Local Navigation System

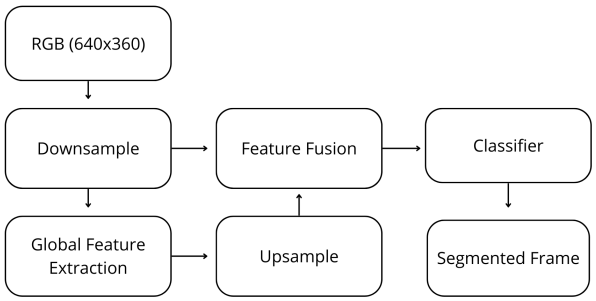

Fig. 2. Block diagram of the Reduced Fast-SCNN architecture.

Building upon the original Fast-SCNN architecture [5], we developed a modified version specifically optimized for lane detection in industrial settings. The original Fast-SCNN was designed for high-speed semantic segmentation on mobile devices; however, in our application, the focus is shifted from segmentation accuracy toward achieving real-time inference on limited computing hardware. The overall structure of the Reduced Fast-SCNN is shown in Fig. 2.

1) Downsampling: The Reduced Fast-SCNN begins with a downsampling stage that reduces the input image to a fixed resolution of 360×640 pixels (width × height). To further reduce computational complexity, the number of convolutional channels is decreased. While the original Fast-SCNN employs 64 channels after downsampling, our modified version uses only 32 channels. This reduction effectively lowers the processing load, with a marginal trade-off in segmentation precision—an acceptable compromise for static and structured industrial environments.

2) Global Feature Extraction: Following the same principle, the global feature extraction stage also reduces the number of channels. In the original architecture, this stage uses 128 channels, whereas in the Reduced Fast-SCNN it is limited to 64 channels. Additionally, the pyramid pooling module is replaced with a lightweight average pooling layer. Compared to pyramid pooling, average pooling simplifies the computation and improves inference speed, making it more suitable for CPU-only real-time operation.

3) Upsampling, Feature Fusion, and Classification: The upsampling, feature fusion, and classification stages retain the core structure of the original Fast-SCNN but operate with fewer convolutional channels. The final output of the Reduced Fast-SCNN is a binary segmentation mask representing two classes: lane and non-lane regions. This segmentation output is later utilized to compute a confidence level that influences the vehicle's navigation decisions, particularly in adjusting steering and velocity control.

C. Integration of Both Systems

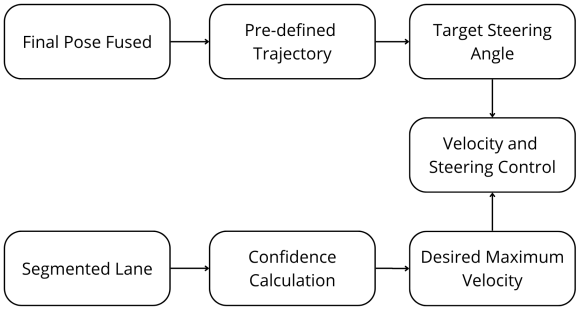

Fig. 3. Integration of the Extended RTAB-Map and Reduced Fast-SCNN for the navigation system.

Figure 3 presents the integrated architecture combining the Extended RTAB-Map for global localization and the Reduced Fast-SCNN for local perception. The navigation controller calculates the heading direction to the next waypoint as:

$$\theta_{\text{direction}} = \tan^{-1}\left(\frac{y_{\text{wp}} - y_{\text{position}}}{x_{\text{wp}} - x_{\text{position}}}\right) - \theta_{\text{orientation}} \tag{2}$$

where x_{wp} and y_{wp} are the coordinates of the next waypoint from the predefined trajectory, x_{position} and y_{position} represent the current vehicle position, and $\theta_{\text{orientation}}$ denotes the current vehicle heading. The resulting $\theta_{\text{direction}}$ is the desired heading angle toward the waypoint.

Using the Bicycle Model [11], the target steering angle is calculated as:

$$\delta_{\text{target}} = \tan^{-1}\left(\frac{2L\sin(\theta_{\text{direction}})}{D_{\text{lookahead}}}\right) \tag{3}$$

where L is the wheelbase (distance between front and rear axles), and $D_{\text{lookahead}}$ is the lookahead distance. The variable δ_{target} represents the desired steering angle for the vehicle.

Next, the system computes a confidence level from the segmented lane detection image. This confidence represents the proportion of detected lane area relative to a predefined region of interest (ROI) and is calculated as:

$$\text{normal} = \frac{\text{area of segmented lane detection}}{\text{area of ROI}} \tag{4}$$

where the numerator denotes the pixel area of the detected lane, and the denominator corresponds to the ROI area. The resulting normal value lies between 0 and 1.

The confidence value is then used to determine the target speed according to:

$$v_{\text{target\_max}} = v_{\text{max}} \cdot \text{normal}^2 \tag{5}$$

where v_{max} is the maximum allowed velocity of the vehicle, and $v_{\text{target\_max}}$ is the confidence-adjusted maximum speed. This ensures the vehicle moves more cautiously when lane confidence is low.

To ensure smooth transitions and suppress sudden speed fluctuations caused by segmentation noise or image flickering, a first-order low-pass filter is applied:

$$v_{\text{target}} = \alpha \cdot v_{\text{target\_max}} + (1 - \alpha) \cdot v_{\text{target}} \tag{6}$$

where α is the smoothing coefficient (set to 0.055), $v_{\text{target\_max}}$ is the instantaneous target speed, and v_{target} is the previously filtered speed. The final output, v_{target}, is used to control the vehicle's longitudinal motion, while δ_{target} regulates steering. Both values are transmitted to the low-level control layer to execute the motion commands in real time.

By combining these two subsystems, the navigation system achieves real-time perception and high-frequency localization, enabling stable and accurate motion control even on limited hardware resources.

III. Results and Discussion

A. Hardware Odometry vs. Extended RTAB-Map

Table I shows pose drift after a 500 m trajectory using odometry only; Table II shows the extended method.

TABLE I
POSE ESTIMATION RESULT: HARDWARE ODOMETRY ONLY

Lap-	X (m)	Y (m)	Error (m)
1	0.28	0.39	0.48
2	0.30	0.43	0.52
3	0.24	0.49	0.55
4	0.18	0.32	0.37
5	0.37	0.41	0.55
Average Error			0.494

TABLE II
POSE ESTIMATION RESULT: EXTENDED RTAB-MAP

Lap-	X (m)	Y (m)	Error (m)
1	0.01	0.02	0.02
2	0.00	0.03	0.03
3	0.01	0.02	0.02
4	0.02	0.01	0.02
5	0.04	0.01	0.04
Average Error			0.026

Figures 4 and 5 visualize the difference in occupancy mapping relative to the ground-truth path (blue line).

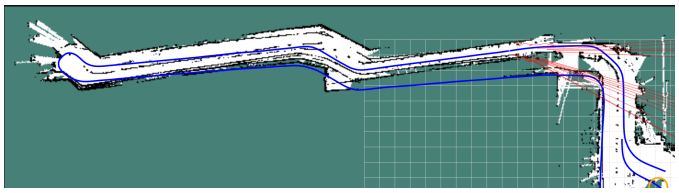

Fig. 4. Occupancy grid map using hardware odometry only.

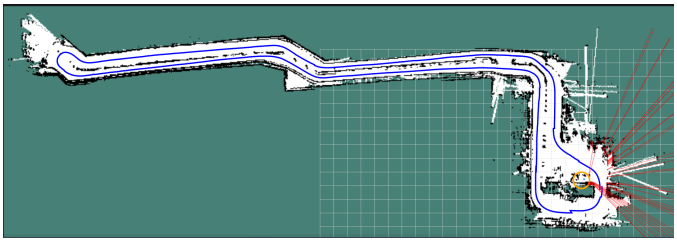

Fig. 5. Occupancy grid map using Extended RTAB-Map.

B. Update Rate: Default vs. Extended RTAB-Map

Update rates were measured via ROS topic frequency tools (Table III). The extended method delivers ∼50 Hz fused pose.

TABLE III
UPDATE RATE: DEFAULT RTAB-MAP VS. EXTENDED RTAB-MAP

Lap-	RTAB-Map (Hz)	Extended RTAB-Map (Hz)
1	1.003	50.029
2	1.002	50.006
3	1.223	49.999
4	1.056	49.999
5	1.332	49.998
Average	1.123	50.006

C. Reduced Fast-SCNN Results

The Reduced Fast-SCNN was trained on a custom dataset of 797 images (size $640 \times 360 \times 3$). The dataset was used in full for training due to its domain specificity. Training used Adam (lr $= 10^{-4}$), batch size 4, for 100 epochs. Sample images are shown in Fig. 6.

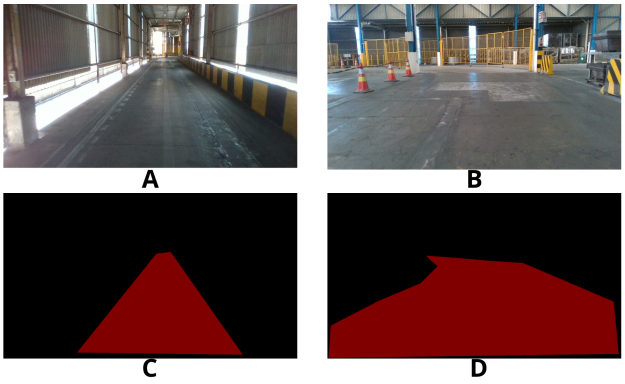

Fig. 6. Sample of the custom dataset (RGB inputs and lane masks).

After training, the model was exported to ONNX. Using ONNX Runtime provided $\sim 2\times$ faster inference [12]. Pre-results for lane detection with an ROI overlay are shown in Fig. 7.

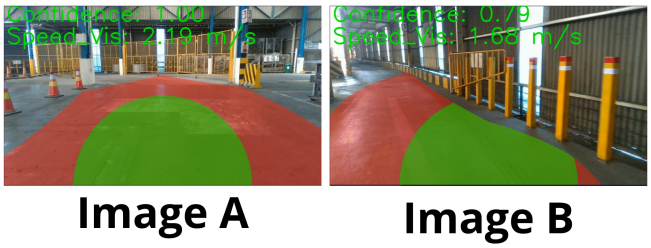

Fig. 7. ROI-based lane detection using the Reduced Fast-SCNN.

Representative outcomes and corresponding velocities are shown in Fig. 8: (A) full straight lane ($v = 2.2$ m/s), (B) narrow straight lane ($v = 1.76$ m/s), (C) sharp turn facing fences ($v = 0.70$ m/s), (D) approaching left turn in a narrow area ($v = 1.59$ m/s), (E) obstacle ahead (stop, $v = 0$), (F) right turn in a narrow area ($v = 1.56$ m/s).

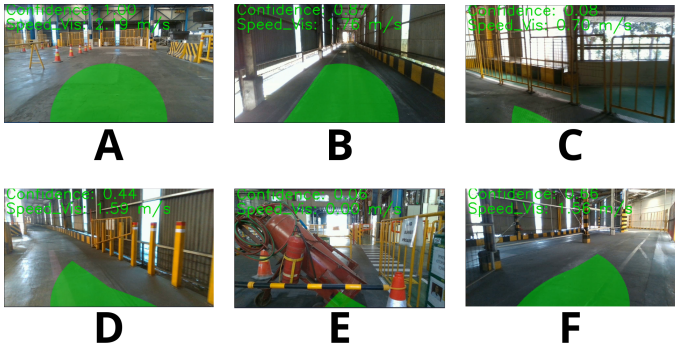

Fig. 8. Lane detection results and resulting target velocities.

D. Inference Time: Original vs. Reduced Fast-SCNN

Inference was benchmarked on an Intel i5 12th Gen CPU with 16 GB RAM and Intel Iris Xe Graphics, using identical $640 \times 360 \times 3$ images (Table IV).

TABLE IV
INFERENCE TIME: ORIGINAL VS. REDUCED FAST-SCNN

Frame Number	Fast-SCNN (ms)	Reduced Fast-SCNN (ms)
1	22.4	7.2
2	29.3	7.7
3	34.2	7.8
4	39.2	7.7
5	23.8	7.4
6	28.1	8.0
Average	29.5	7.63

On average, the Reduced Fast-SCNN is $\sim$3.9$\times$ faster (7.63 ms vs. 29.5 ms), enabling CPU-only real-time operation.

IV. Future Development

Future work includes: (1) improving robustness by adding temporal context via GRU-based modules [13], and (2) learning velocity and steering directly from perception and pose through end-to-end ML.

V. Conclusion

By augmenting RTAB-Map with hardware odometry and Kalman fusion, we achieve centimeter-level pose accuracy with a 50 Hz update rate on a CPU-only platform. The Reduced Fast-SCNN attains sub-10 ms inference, making the combined navigation stack efficient and suitable for industrial autonomous vehicles.

VI. Acknowledgment

This work was supported by Indonesia Smelting Technology (IST), Manufaktur Robot Industri (MRI), and ITS Robotics. The authors thank the supporting teams for their assistance. We also acknowledge ChatGPT and GitHub Copilot for writing support.

References

[1] R. Kazemi, R. Haidarimoghadam, M. Motamedzadeh, R. Golmohammadi, A. Soltanian, and M. R. Zoghipaydar, "Effects of shift work on cognitive performance, sleep quality, and sleepiness among petrochemical control room operators," *Journal of Circadian Rhythms*, vol. 14, p. 1, 2016. [Online]. Available: https://jcircadianrhythms.com/articles/10.5334/jcr.134

[2] M. Labbé and F. Michaud, "Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," in *Journal of Field Robotics*, vol. 36, no. 2. Wiley Online Library, 2019, pp. 416–446.

[3] B. Clipp, J. Lim, J.-M. Frahm, and M. Pollefeys, "Parallel, real-time visual slam," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010, pp. 3961–3968.

[4] M. I. Zazuli, R. Dikairono, D. Purwanto, Muhtadin, and M. L. Hakim, "Sensor fusion system for localization of autonomous car," in *2024 International Conference on Green Energy, Computing and Sustainable Technology (GECOST)*, 2024, pp. 1–5.

[5] A. C. Poudel, S. Liwicki, and R. Cipolla, "Fast-scnn: Fast semantic segmentation network," 2019.

[6] M. L. R. Lagahit and M. Matsuoka, "D-fast-scnn + combo loss: Improved road marking extraction on mobile lidar sparse point cloud-derived images," in *IGARSS 2023 - 2023 IEEE International Geoscience and Remote Sensing Symposium*, 2023, pp. 5684–5687.

[7] S. A. Khan Tareen and R. H. Raza, "Potential of sift, surf, kaze, akaze, orb, brisk, agast, and 7 more algorithms for matching extremely variant image pairs," in *2023 4th International Conference on Computing, Mathematics and Engineering Technologies (iCoMET)*, 2023, pp. 1–6.

[8] A. Vinay, A. S. Rao, V. S. Shekhar, A. C. Kumar, and K. N. B. Murthy, "Feature extraction using orb-ransac for face recognition," *Procedia Computer Science*, vol. 70, pp. 174–184, 2015, presented at 4th International Conference on Eco-friendly Computing and Communication Systems (ICECCS 2015).

[9] J. Zhang, Y. Yao, and B. Deng, "Fast and robust iterative closest point," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3450–3466, 2022.

[10] F. Dellaert, "Factor graphs and gtsam: A hands-on introduction," *Georgia Institute of Technology*, 2012, available at https://gtsam.org.

[11] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. Boston, MA: Springer, 2011.

[12] F. Xu, "Faster scalable ml model deployment using onnx and open source tools," in *2020 IEEE Infrastructure Conference*, 2020, pp. i–i.

[13] T.-W. Yu, M. A. Sarwar, Y.-A. Daraghmi, S.-H. Cheng, T.-U. İk, and Y.-L. Li, "Spatiotemporal activity semantics understanding based on foreground object segmentation: icounter scenario," *IEEE Access*, vol. 10, pp. 57 748–57 758, 2022.